LearnDeep

Technical Reference

Subhodeep Mondal

October 24, 2026

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Motivation . . . 1
- 1.3 Scope . . . 1
- 1.4 Organization of This Document . . . 2

2 Tensor System **3**

3 Operations **4**

4 Kernels **5**

5 Automatic Differentiation **6**

6 Neural Network Components: Layers **7**
- 6.1 Layer . . . 7
- 6.2 Layer: Dense . . . 8
  - 6.2.1 Dense: Constructor . . . 9
  - 6.2.2 Dense: Overloaded () operator . . . 9
  - 6.2.3 Dense: forward() backward() calculate() . . . 10
  - 6.2.4 Dense in practice . . . 11

7 Neural Network Components: Losses **13**
- 7.1 Loss Functions: Mean Squared Error (MSE) . . . 14
  - 7.1.1 MSE: Differentiation . . . 14
  - 7.1.2 MSE: Putting things together . . . 14
- 7.2 Loss Function: Categorical Cross Entropy (CCE) . . . 15
  - 7.2.1 CCE: Differentiation . . . 15
  - 7.2.2 CCE: Putting things together . . . 15
  - 7.2.3 CCE: In practice . . . 16

8 Neural Network Components: Optimizers **17**
- 8.1 Gradient Descent: GD . . . 18
  - 8.1.1 Batch Gradient Descent: BGD . . . 18
  - 8.1.2 Mini Batch Gradient Descent: MGD . . . 18
  - 8.1.3 Stocastic Gradient Descent: SGD . . . 19
- 8.2 Stocastic Gradient Descent . . . 19
- 8.3 SGD with momentum . . . 20

9 Neural Network Model Training and Inference **21**
9.1 The Model Class 22
9.1.1 Model Construction and Compilation 22
9.1.2 Model Training 23
9.1.3 Model Inference 24
9.2 Numerical Verification 25

Chapter 1

Introduction

1.1 Overview

LearnDeep is an educational deep learning framework developed from scratch to explore the fundamental concepts, algorithms, and engineering principles behind modern deep learning systems.

The primary objective of this project is to understand how a deep learning framework operates internally, from tensor computations and numerical kernels to automatic differentiation, neural network layers, and model training.

Rather than treating a deep learning framework as a black box, LearnDeep aims to make its underlying mechanisms explicit through a modular implementation and a focus on mathematical correctness.

1.2 Motivation

Modern deep learning frameworks provide high-level interfaces for constructing, training, and evaluating neural networks. However, many of the mechanisms responsible for these operations remain hidden behind their public APIs.

Developing LearnDeep provides an opportunity to investigate these mechanisms directly, including:

- Tensor representation, indexing, and memory ownership.
- Numerical operations, broadcasting, and CPU and GPU kernel execution.
- Computational graphs and reverse-mode automatic differentiation.
- Neural network layers, loss functions, and gradient propagation.
- Model training, optimization, and execution management.

Implementing these components from the ground up provides practical insight into the relationship between mathematical operations, software architecture, and hardware-level computation.

1.3 Scope

LearnDeep is intended primarily as a learning and experimentation platform. Its implementation prioritizes understanding, modularity, and correctness while exploring the design of a deep learning framework.

The technical reference documents the framework's architecture, public interfaces, mathematical foundations, and important implementation decisions.

It is organized around the major components of the system, allowing readers to understand both how individual components work and how they interact during model execution and training.

1.4 Organization of This Document

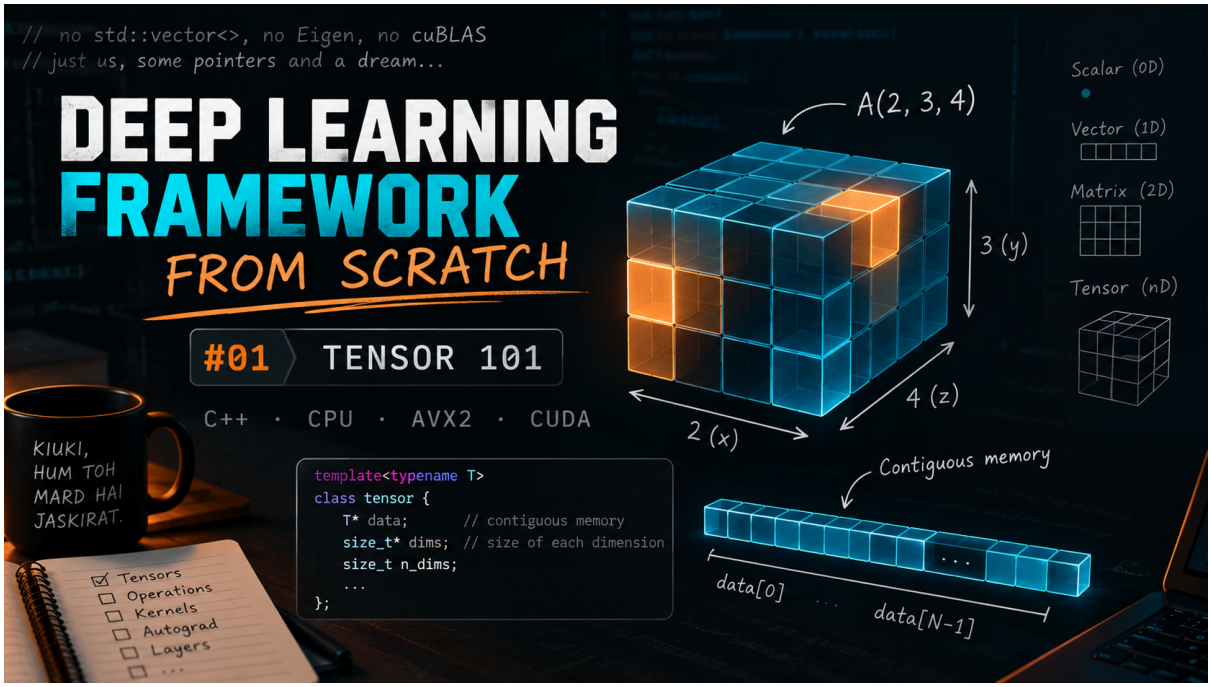
The remainder of this document is organized as follows:

- **Tensor System:** Tensor representation, shape semantics, data layout, and ownership.
- **Operations and Kernels:** Numerical operations, broadcasting, and backend execution.
- **Automatic Differentiation:** Computational graphs, gradient propagation, and backward execution.
- **Neural Network Components:** Layers, loss functions, and model construction.
- **Training:** Forward execution, optimization, and training workflows.

These sections describe the design and behavior of the implemented system, including relevant mathematical definitions and validation methods.

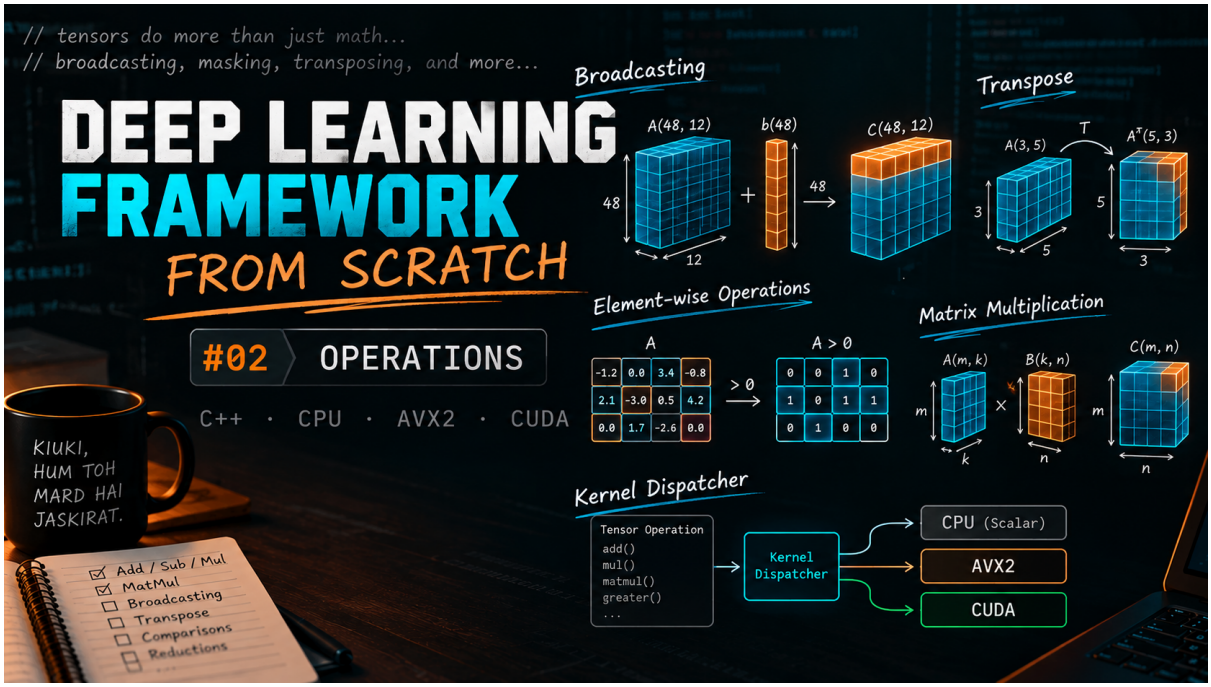
Chapter 2

Tensor System



Chapter 3

Operations

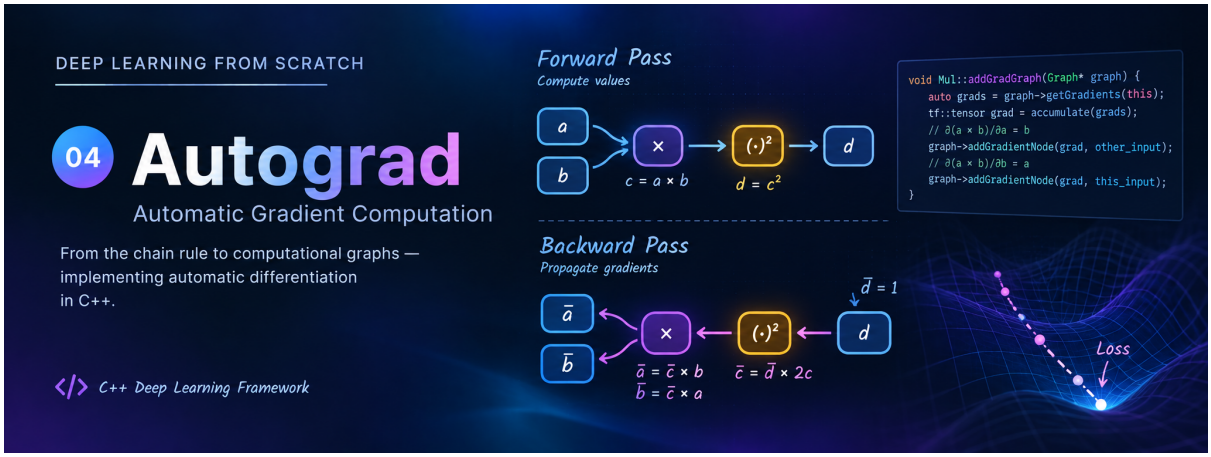


Chapter 4

Kernels

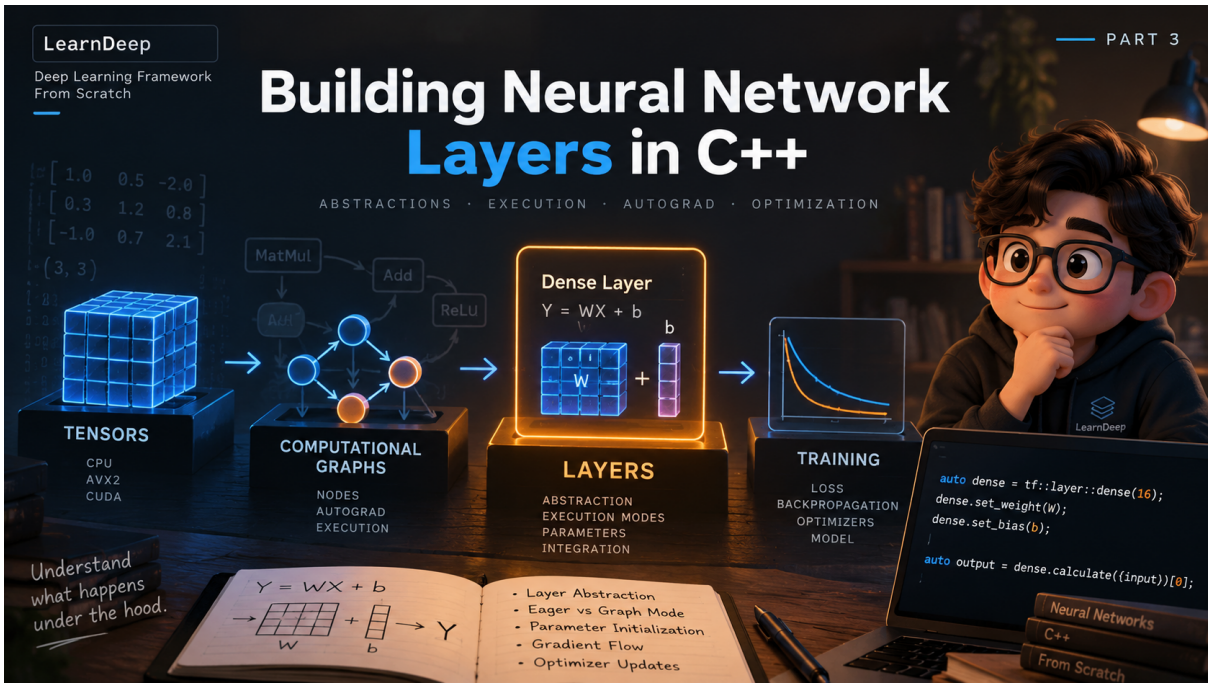
Chapter 5

Automatic Differentiation



Chapter 6

Neural Network Components: Layers



6.1 Layer

A layer is a high-level abstraction that encapsulates a sequence of mathematical operations into a reusable component. It allows us to construct complex neural network architectures without explicitly writing every underlying operation or manually defining the backward graph for each layer.

This abstraction separates model architecture from the mathematical details of individual operations. As a result, we can modify the architecture and its hyperparameters while relying on the framework to handle the underlying computations and automatic differentiation.

Let's now examine how layers are implemented in this framework.

The Layer class is an abstract base class that defines the common interface for derived layers. Each concrete layer implements the required functions:

the layer class is a abstruct class i.e can't be instantiated then later layers are inheriting and overriding the functions like

- calculate: Calculates the forward pass only in eager mode
- forward: Constructs the layer's computation in the current graph context for graph-mode execution.
- backward: Retrieves the gradients required by the layer's parameters and registers the corresponding optimizer update operations. It does not explicitly construct the backward graph for the layer's mathematical operations; those gradients are handled by the autograd system.
- basic constructor and destructors and some axulary functions to intercept the parameters or hyperparameters.

Let's see it in practice

```
class Layer {
public:
    virtual ~Layer() = default;

    virtual std::vector<tf::tensor>
    calculate(std::vector<tf::tensor> inputs) = 0;
    virtual std::vector<tf::tensor*>
    forward(std::vector<tf::tensor> inputs) = 0;
    virtual void backward() = 0;
};
```

Listing 6.1: Creating layer class

Now lets take a look at each layer and how they're constructed

6.2 Layer: Dense

Mathematically Dense layer is very simple, it does matmul bias add, which is
$Y = X@W + b$

- X: input temsor of rank 2. axis-1 is feature axis-2 is batch or sample size.
- W: is weights tensor of rank 2 axis-1 is Dense unit/no or neuron axis-2 is no of incoming features.
- b: is a vector or matrix of rank 1. axis-1 is no of dense unit/ no of neuron.
- Y: output tensor of rank 2. axis-1 is no of feature and axis-2 is batch or sample size.

The idea behind Dense layer is do a linear transformation on input with weights tensor and add bias to it.

If we wanted to implemente a dense layer without any layer abstruction it would look something like

```
#include <tensor.h>

tf::tensor X, W, B, Y;

X.tf_create(18, 512); // no_of_input_feature(18), and sample_size(512)
W.tf_create(16, 18);  // dense_unit(16), and no_of_incoming_feature(18)
B.tf_create(18); // dense_unit(16);

Y = X.matmul(W).add(B);
```

Listing 6.2: Dense Function

First of all we can we that but if we think from bigger perspective then it would be very clear that. This approch is not that easy to scale. If we get to the training phase then we need to mannually handle gradients and update them. So that is the reason behind abstruction.

No Lets take a look at how to properly make the abstruction

6.2.1 Dense: Constructor

```
// file dense.hpp
#include <layer.hpp>

Class Dense : protected Layer{
    Dense(unsigned unit): dense_unit(unit){}

    std::vector<tf::tensor *>& operator(std::vector<tf::tensor>
   input_tensors);
    std::vector<tf::tensor> calculate(std::vactor<tf::tensor> inputs)
   override;
    std::vector<tf::tensor *>& forward(std::vector<tf::tensor> inputs,
   unsigned const batch_size) override;
    std::vector<tf::tensor *>& backward(Optimizer *const optimizer)
   override;
    ...
    // few other internal methods/functions
    ...
}
```

Listing 6.3: Dense constuctor and basic structure

simple easy peasy no rocket science. constructor is expecting the no of neuron for this layer. and assigning the vaue to its internal variable and few other internal flags, weights, bias initialization methods are set. Which is not shown here.

6.2.2 Dense: Overloaded () operator

```

// file dense.cpp
#include "dense.hpp"

const std::vector<tf::tensor *> &
Dense::operator()(std::vector<const tf::tensor> input_tensors){
    if(input_tensor.size() == 1){
        weights.tf_create();
        bias.tf_create();
        layer_outputs.push_back(tf::tensor(tf_float64));
    } else {
        // throw error, Dense layer expects only one input tensor
    }

}
```

Listing 6.4: Dense () overloaad

The overloaded operator() provides a convenient interface for connecting layers. During this stage, the layer prepares its parameters and output handles as required by the framework.

The initialization is lazy because the layer may not have enough information to initialize all its internal state when the constructor is called. The first build or forward operation provides the necessary context, such as the input shape and batch size.

The output handle is particularly important: it allows subsequent layers to reference the output of this layer without manually connecting every underlying mathematical operation.

Why use vectors of tensors instead of a single tensor? Although a Dense layer accepts one input tensor and produces one output tensor, the base interface supports layers that may consume multiple inputs or produce multiple outputs. This keeps the layer interface general without forcing every layer to use the same input/output cardinality.

6.2.3 Dense: forward() backward() calculate()

```
1
2  // file dense.cpp
3  #include "dense.hpp"
4
5  const std::vector<tf::tensor *> &
6  Dense::forward(std:;vector<const tf::tensor *> &inpyt, unsigned
       batch_size){
7
8        this->weight.reshape(arr);
9        this->training_weight.tf_create(arr, tf_float64);
10       this->grad_weight.tf_create(arr, tf_float64);
11
12       ...
13       // initialize bias
14       // initialize outputs
15       ...
16
17       training_output.push_back(training_input.matmul(training_weights)
       .add(training_bias));
18
19       return training_ouput;
20
21 }
22
23
24
25 const std::vector<tf::tensor *> &
26 Dense::calculate(std:;vector<const tf::tensor *> &inpyt, unsigned
       batch_size){
27 initializeParameters();
28 std::vector<tf::tensor> output;
29
30 output.emplace_back(inputs[0].matmul(weights).add(bias));
31
32 return output;
33 }
34
35 Dense::backward(Optimizer *optimizer){
36
37   if (!this->isGradRecorded) {
38     Graph *g = GraphManager::instance().getCurrentGraph();
39     graph_weight = g->getGradTensor(training_weight);
40     grad_bias =  g->getGradTensor(training_bias);
41     grad_input = g->getGradTensor(training_inputs[0]);
42     grad_output = g->getGradTensor(training_output[0]);
43
44     optimizer->createParameterUpdateGraph(training_weight,
       updated_weight, grad_weight);
```

```
45      optimizer->createParameterUpdateGraph(training_bias, updated_bias,
        grad_bias);
46
47      this->isGradRecorded = true;
48    }
49  }
```

Listing 6.5: Dense calculate

Lets discuss one at a time:

forward()

This function implements the graph-mode forward path and is primarily invoked during model training through fit().

At this stage, the framework has access to the execution context and the relevant input shapes. The layer can initialize its parameters, prepare the required tensors, and construct the computation using the framework's primitive operations.

The graph context is managed by the model's training procedure, so the layer does not need to create a separate graph context for itself.

The important distinction is that forward() constructs the computation graph; it does not necessarily evaluate the tensor operations immediately.

calculate()

This function implements the eager-mode forward path and is primarily used by predict() when immediate inference is required.

Unlike graph construction, eager execution evaluates the underlying operations as they are invoked and returns the resulting output tensors.

backward()

It retrieves the gradients associated with the weights, biases, inputs, and outputs from the current graph context, as required by the implementation. It then registers the parameter update operations with the optimizer.

The gradients of the layer's mathematical operations are computed by the autograd system. The layer's backward() method does not manually derive those gradients; instead, it exposes the relevant gradient tensors and prepares the optimizer update path.

The isGradRecorded flag prevents these registrations from being repeated unnecessarily within the intended graph lifecycle.

6.2.4 Dense in practice

```
1  // file dense_test_01.cpp
2  #include <tensor.h>
3
4  void main(){
5      tf::tensor inputs, W, B, ouputs;
6
7      inputs.tf_create(tf_float64, 19, 1000);
8      W.tf_create(tf_float64, 16, 19); // tensor to seed weight
9      B.tf_create(tf_tf_float64, 16);  // tensor to seed bias
10
11     input.tensor_of(-2.0, 3.0);
12     W.tensor_of(-1.0, 1.0);
```

```
    B.tensor_of(0.0, 0.0);

    auto dense = tf::layer::dense(16); // 16 is dense unit a.k.a no of
    neurons
    dense.set_weight(W);
    dense.set_bias(B);

    output = dense.calculate({inputs})[0]; // output holds the output
    from dense layer

}
```

Listing 6.6: Dense in practice

Dense Training

backward(), is already covered so, we know how it is done in code but let's touch upon mathematical construct of Dense backward

Weight updation

$$\bar{W} \leftarrow W - \nu * \delta W$$

$$\delta W = ZX^T$$

Bias updation

$$\bar{B} \leftarrow B - \nu * \delta B$$

$$\delta W = X$$

Gradient propagation

$$\delta X = W^T Z$$

The input gradient is propagated to the preceding layer; it is not updated using the learning rate. The optimizer updates trainable parameters, while autograd propagates gradients through the computational graph.

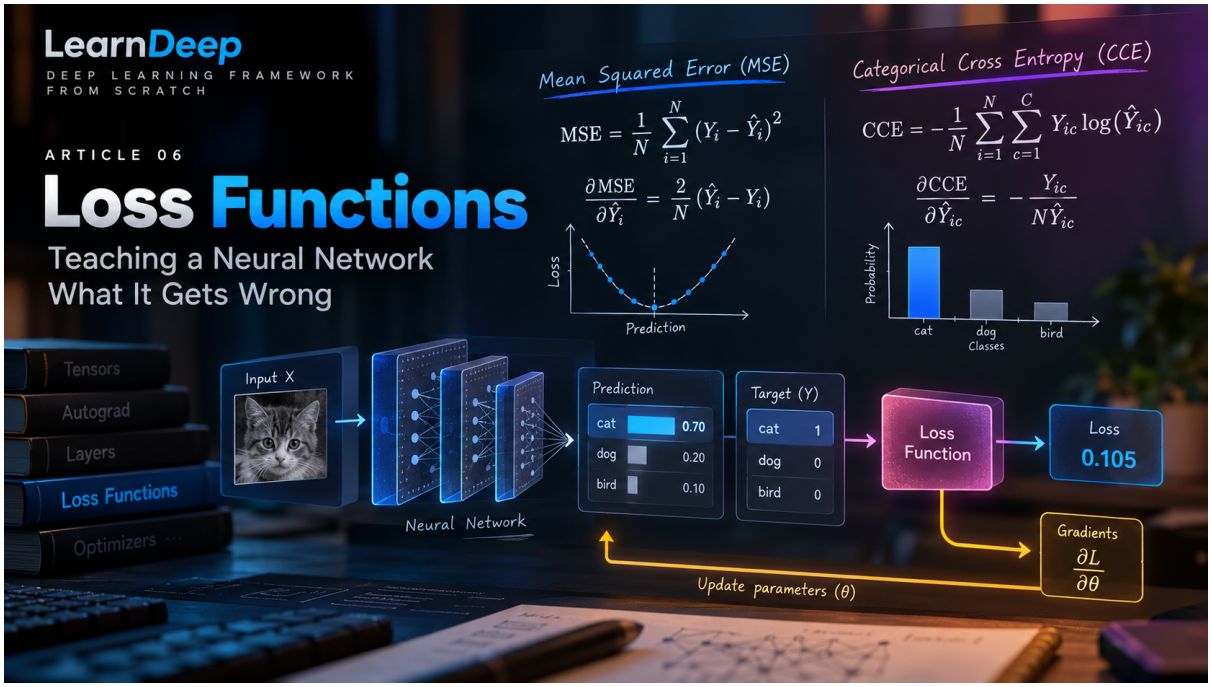
Where,

Z is incoming grad

X is input

Chapter 7

Neural Network Components: Losses



From the previous chapters, we've established the core components of our framework, including tensors, operations, autodiff, and neural network components such as layers.

So far, our framework is good at performing mathematical computations through forward and backward passes. But how does it know whether its predictions are correct? More importantly, how does it know whether training is making progress?

Our model's trainable parameters are initialized, and gradients can be calculated through the computational graph. But we still need a way to measure the difference between the model's predictions and the expected outputs.

That's where loss functions come in.

In supervised learning, a loss function measures the discrepancy between predicted outputs and their corresponding targets. During training, we typically aim to minimize this loss. For each batch, the model produces predictions during the forward pass, and the loss function evaluates how far those predictions are from the actual targets.

Let's see how this works in practice.

7.1 Loss Functions: Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(Y_i - \hat{Y}_i)^2$$

where N is the number of elements over which the squared errors are averaged, Y_i is the target, and $\hat{Y}_i$ is the predicted output.

For each element, we subtract the prediction from the target and square the resulting difference. We then calculate the mean of these squared errors over the selected elements.

7.1.1 MSE: Differentiation

Although our autodiff engine calculates the gradients automatically, let's look at the derivative to understand what happens during backpropagation.

Differentiating MSE with respect to a prediction gives:

$$\frac{\partial \text{MSE}}{\partial \hat{Y}_i} = \frac{2}{N}(\hat{Y}_i - Y_i)$$

This gradient measures how the loss changes with respect to each predicted value. Our autodiff engine propagates it through the computational graph to calculate gradients for the model's trainable parameters.

7.1.2 MSE: Putting things together

Okey, now as we've sorted out the math part, we can now focus on the implementation. If we think deeply losses share common traits from their bigger sibling layers. They have few differences

- 1. Don't have trainable parameter
- 2. Outputs a scaler value rather than tensor.

So, we can follow same structure like Layers

```cpp
// file loss.hpp

class Loss {
public:
    virtual ~Loss() = default;

    virtual float calculate(std::vector<tf::tensor> inputs) = 0;
    virtual void forward(std::vector<tf::tensor> inputs) = 0;
    virtual void backward() = 0;
};
```

Listing 7.1: Creating loss class

Although the loss function does not need to implement differentiation manually, its backward interface can provide access to the gradients recorded by the computational graph. This is useful for debugging and inspecting the intermediate gradients.

Let's see how MeanSquaredError(MSE) is layed out.

```cpp
// file mse.cpp
#include <mse.hpp>

    void MSE::forward(std::vector<tf::tensor> inputs) {
      if (inputs.size() == 1) {
```

```
        training_inputs.push_back(inputs[0]);
        ...
        // initialize the intermediate tensors
        ...

        differences = target_ouput.sub(training_inputs[0]);
        loss_tensor = differences.pow(2).scale(1.0/batch_size);
      }
    };
    void MSE::backward(){
        if (!this->isGradRecorded) {
            this->isGradRecorded = true;
            Graph *g = GraphManager::instance().getCurrentGraph();
            grad_training_inputs = g,getGradTensor(trining_inputs[0]);
        }
    };
```

Listing 7.2: Creating MSE class

but sadly I've don't have any standalone configuration to showcase how a user can use losses. Most of its usage are inside model. As of now simple eager execution is not ready yet.

7.2 Loss Function: Categorical Cross Entropy (CCE)

While MSE is commonly used for regression tasks, Categorical Cross Entropy (CCE) is commonly used for classification tasks. It measures how well the predicted class probabilities match the target distribution.

For one-hot encoded targets, CCE is defined as:

$$\text{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} Y_{ic}\log(\hat{Y}_{ic})$$

where N is the batch size and C is the number of classes.

For each sample, we multiply the target value for each class by the logarithm of its predicted probability, sum over the classes, and average over the batch. The negative sign ensures that minimizing the loss encourages the model to assign higher probabilities to the correct classes.

7.2.1 CCE: Differentiation

$$\frac{\partial CCE_{\text{batch}}}{\partial(\hat{Y})} = -\frac{1}{n}\frac{Y}{\hat{Y}}$$

7.2.2 CCE: Putting things together

```
// file cce.cpp
#include <cce.hpp>

    void CCE::forward(std::vector<tf::tensor> inputs) {
      if (inputs.size() == 1) {
        training_inputs.push_back(inputs[0]);
        ...
        // initialize the intermediate tensors
        ...

        log_differences = target_inputs[0].mul(training_inputs[0].log()
   );
```

```
        loss_tensor = log_differences.scale(1.0/batch_size);
      }
    };
    void CCE::backward(){
        if (!this->isGradRecorded) {
            this->isGradRecorded = true;
            Graph *g = GraphManager::instance().getCurrentGraph();
            grad_training_inputs = g,getGradTensor(trining_inputs[0]);
        }
    };
```

Listing 7.3: Creating CCE class

7.2.3 CCE: In practice

Like MSE I've don't have eager mode calculate() is still not ready. So I don't have much ex. to showcase.

Chapter 8

Neural Network Components: Optimizers



We've already established the basic workings of an end-to-end neural network. We've seen how tensors, operations, graphs, layers and losses come together to perform a sequence of mathematical operations that produce the intended output.

However, measuring how far the prediction is from the desired result is only part of the process. As discussed in Chapter 7, the loss function quantifies the prediction error. We still need a way to minimize this error by updating the model parameters. This is where optimizers come into play.

During initialization, the model parameters are typically assigned values using an initialization strategy. These initial values generally do not produce satisfactory predictions. During training, the optimizer uses the gradients calculated during backpropagation to update the model parameters, helping the network minimize its loss.

We've already seen that the model parameters are initialized randomly. So at first it is going to give very incorrect output. Now it is optimizer which makes changes to the model parame-

ters based on the gradients calculated during backward pass that pushes model parameters to calculate output close to desired target. Lets see how it works.

8.1 Gradient Descent: GD

Gradient descent is one of the fundamental optimization algorithms used to train neural networks. It updates model parameters in the direction opposite to the gradient of the loss function.

Compared with second-order methods such as Newton-Raphson, standard gradient descent is generally simpler to implement and requires less memory because it does not need to compute and store the full Hessian matrix. This makes it a practical choice for large neural networks.

$$W_{\mathrm{t}+1} = W_{\mathrm{t}} - \eta * \partial W_t$$

from the above example we can easily tell what a optimizer suppose to do. For a model parameter W we're updating it's value by subtracting partial difference times η with original value. η is learning rate.

Learning Rate The learning rate, denoted by η, controls the step size of each parameter update.

If the learning rate is too large, the optimizer may overshoot the minimum, oscillate around it, or even diverge. If it is too small, convergence may become unnecessarily slow.

Choosing an appropriate learning rate is therefore critical to effective training. In practice, the learning rate may also be adjusted during training using a learning-rate schedule.

We're not going to work on a single sample. So working with a large sample size. Based on how we can tackle a large sample size and update model parameter values there are three distinct GD stratigies in place.

- Batch Gradient Descent(BGD)
- Mini Batch Gradient Descent(MGD)
- Stocastic Gradicnt Descent(SGD)

Now, Lets take a look at each of them.

8.1.1 Batch Gradient Descent: BGD

The gradient is calculated using the entire training dataset, and the parameters are updated once after the full dataset has been evaluated.

Pros: The gradient represents the entire dataset, giving a consistent update direction for the current parameter values.

Cons: Each update can be computationally expensive for large datasets.

8.1.2 Mini Batch Gradient Descent: MGD

The training dataset is divided into smaller batches. The model processes each batch and updates its parameters after each batch.

Pros: Balances computational efficiency and gradient stability. It also works well with vectorized CPU and GPU operations.

Cons: The gradient is an estimate of the full-dataset gradient, so updates can fluctuate between batches.

8.1.3 Stocastic Gradient Descent: SGD

In its strict definition, the gradient is calculated using one randomly selected training sample per update.

Pros: Updates are inexpensive and frequent. The noise in the gradient can sometimes help the optimizer move away from saddle regions or shallow local minima.

Cons: Updates can fluctuate considerably, making convergence less smooth.

8.2 Stocastic Gradient Descent

I've briefly touched SGD in the previous section but lets dig deeper into this algorithm. So I've already established how SGD works Mathematically. Let's see how it is integrated with the framework.

```
// file optimizer.hpp
class Optimizer{
    Graph *optimizer_graph;

  virtual void createParameterUpdateGraph(tf::tensor param,
                                          tf::tensor &update_param,
                                          tf::tensor gradient_param) =
   0;

  void executeOptimizer();
}
```

Listing 8.1: Creating Optimizer class

```
// file optimizer.cpp
#include "optimizers.hpp"
#include <core/graph/graph_framework.hpp>

Optimizer::Optimizer() { this->optimizer_graph = new Graph(); }
void Optimizer::executeOptimizer() { this->optimizer_graph->compute();
   }
```

Listing 8.2: Defining Optimizer class

So, Optimizer is a abstruct class which works as base class for other optimizers.

The createParameterUpdateGraph() subrountine is something new. Mostly for industry standard frameworks optimizers works in eager mode, in my case it is operating in graph mode. Why not to attempt something differet.

```
// file sgd.cpp
#include "optimizers.hpp"
#include <core/graph/graph_manager.hpp>

SGD::SGD() : learning_rate(0.01) {}

void SGD::createParameterUpdateGraph(tf::tensor param, tf::tensor &
   update_param,
                                     tf::tensor gradient_param) {
```

```
9
10    GraphManager::instance().pushGraph(this->optimizer_graph);
11    scaler_param.push_back(tf::tensor());
12
13    std::vector<unsigned> dims;
14    for (unsigned i = 0; i < param.getNoOfDimensions(); i++)
15      dims.push_back(param.getDimensions()[i]);
16    scaler_param.back().tf_create(dims, param.dt_type);
17
18    scaler_param.back() = gradient_param.scale(learning_rate);
19    update_param = param.sub(scaler_param.back());
20
21    GraphManager::instance().popGraph();
22  }
23
24  void SGD::updateLearningRate(std::float64_t learning_rate) {
25    this->learning_rate = learning_rate;
26  }
```

Listing 8.3: Defining SGD class

Here we can see GraphManager in action. By pushing the optimizer graph onto the graph manager, the operations used to calculate parameter updates are recorded in the appropriate graph.

This graph is constructed during the setup of the training process for each trainable parameter. Once a training batch has completed its forward and backward passes, the training loop executes the optimizer graph to calculate the parameter updates.

This separates the construction of the update operations from their execution during training.

Although vanilla SGD is simple and computationally efficient, it has some limitations. Its updates can fluctuate significantly, and it may converge slowly in regions where the loss landscape has poorly conditioned curvature.

Several optimizer variants address these issues by incorporating additional information into the parameter updates. One of the simplest and most important extensions is SGD with momentum, which we will explore next.

8.3 SGD with momentum

Chapter 9

Neural Network Model Training and Inference



So far, we have developed the fundamental components required to construct and train a neural network, including tensors, layers, loss functions, optimizers, and automatic differentiation.

The next step is to bring these components together into a complete training and inference workflow.

In this chapter, we will explore the public model API exposed by LearnDeep and understand how it orchestrates the underlying components. We will examine model construction, compilation, training, and inference, and discuss how the implementation can be verified against independently generated reference results using NumPy.

The objective is not to revisit the implementation details of each component, but to understand how they interact to form a functional neural network framework.

9.1 The Model Class

The Model class provides the high-level interface through which users define and execute a neural network. It abstracts the underlying tensor operations, computational graph, automatic differentiation, loss calculation, and parameter optimization behind a small set of public API functions.

The following listing presents the principal interface of the Model class.

```
class Model {
  void compile(const OptimizerType optimizer_type,
               const LossType loss_type);

  tf::history_container fit(
      const std::vector<tf::tensor>& inputs,
      const std::vector<tf::tensor>& outputs,
      const std::vector<tf::tensor>& validation_data = {},
      unsigned epochs = 10,
      unsigned batch_size = 1,
      std::vector<std::shared_ptr<Callback>> callback = {},
      unsigned verbose = 0,
      bool training = true);

  std::vector<tf::tensor> predict(
      std::vector<tf::tensor> inputs);

  void shuffle(bool shuffle);

  tf::loss getModelLoss(
      Tensor<std::float64_t>* output_tensor);
}
```

Listing 9.1: Public interface of the Model class

The API separates the definition of a model from the operations performed on it. Model construction establishes the relationship between input and output tensors, while compilation configures the loss function and optimizer. The fit() method coordinates training, and predict() provides an interface for evaluating the model on new inputs.

The remaining methods support training configuration and loss-related operations.

Let us examine how these responsibilities are implemented.

9.1.1 Model Construction and Compilation

Many neural network libraries provide a sequential interface in which layers are arranged in a predefined order. Each layer receives the output of the preceding layer and passes its own output to the next layer.

LearnDeep instead represents a model through its input and output tensors and the computational dependencies connecting them. This approach is not restricted to a sequential arrangement of layers.

For example, consider the following model construction:

```
tf::model mymodel({x}, dense_output);

mymodel.shuffle(false);

mymodel.compile(OptimizerType::SGD,
                LossType::squared_error);
```

Listing 9.2: Constructing and compiling a model

Here, x represents the model input, while dense_output represents the output tensor produced by the network. The model is constructed by associating these tensors, rather than by explicitly passing an ordered collection of layers to the Model constructor.

During model construction, LearnDeep traverses the dependencies of the declared output tensors to determine whether they can be traced back to the declared inputs. If the required dependencies cannot be resolved, the model construction is rejected.

This process establishes the computational path required to evaluate the model. It also provides the foundation for executing the forward computation and, during training, propagating gradients through the relevant operations.

The distinction between a sequential arrangement and a computational graph becomes particularly important when a model contains multiple inputs, branching paths, or operations that combine outputs from different parts of the network.

The supported graph structures ultimately depend on the operations and execution strategies implemented by LearnDeep.

Compilation Once the model has been constructed, compile() configures the components required for training.

The optimizer type determines the parameter update strategy, while the loss type determines how the model's predictions are evaluated against the target outputs.

For example, the configuration shown above selects stochastic gradient descent (SGD) as the optimizer and squared error as the loss function.

Based on these selections, LearnDeep creates the corresponding optimizer and loss instances. This separates the choice of training configuration from the implementation of the training loop itself.

The optimizer and loss function serve different purposes. The loss function quantifies the prediction error, whereas the optimizer uses the resulting gradients to update the trainable parameters.

Training Data Shuffling The shuffle() method controls whether the training samples are shuffled before being divided into mini-batches.

When shuffling is disabled, the samples retain their original order. When enabled, their order is randomized according to the implementation.

In the example above, shuffle(false) preserves the original sample order.

Shuffling is useful during training because it reduces dependence on the original ordering of the training samples. However, it does not change the underlying model architecture or the mathematical definition of its operations.

9.1.2 Model Training

Training is the process through which a neural network adjusts its trainable parameters to reduce the discrepancy between its predictions and the target outputs.

The fit() method provides the interface for executing this process.

```
unsigned epochs = 10;
unsigned batch_size = 128;

auto history = mymodel.fit(
    {input},
    {target_output},
    {},
    epochs,
    batch_size,
    callback);
```

Listing 9.3: Training a model

The training inputs and corresponding target outputs are provided as collections of tensors. The epochs argument specifies how many complete passes are performed over the training dataset, while batch_size determines the number of samples processed in an individual mini-batch.

During training, the framework coordinates the following operations:

1. **Mini-batch selection:** The training samples are divided into batches according to the configured batch size and shuffling policy.
2. **Forward propagation:** The input tensors are passed through the computational graph to calculate the model predictions.
3. **Loss calculation:** The predictions are compared with the corresponding target outputs using the configured loss function.
4. **Backward propagation:** Automatic differentiation computes the gradients of the loss with respect to the trainable parameters.
5. **Parameter optimization:** The optimizer uses the calculated gradients to update the model parameters.
6. **Iteration:** The process continues for the remaining mini-batches and repeats until the requested number of epochs has been completed.

The process connects the components developed in the previous chapters. Tensor operations perform the forward computation, the loss function evaluates the predictions, automatic differentiation calculates gradients, and the optimizer updates the parameters.

The fit() method therefore acts as the coordinator of the training workflow rather than implementing each of these operations independently.

Callbacks and Training History Callbacks provide a mechanism for observing or responding to events during training. Depending on the callback implementation, they can be used to record metrics, monitor training progress, or control the training process.

The fit() method returns a history container that preserves the training information recorded during execution. This allows the user to inspect the training results after the fit() call has completed.

The exact contents of the history depend on the metrics and callback behavior supported by LearnDeep.

The validation data argument can be used to provide additional data for evaluating the model during training, provided that validation evaluation is implemented by the training workflow. Validation data is distinct from the training data because it is used to evaluate model behavior rather than to perform gradient-based parameter updates.

9.1.3 Model Inference

After training, the model can be used to generate predictions for a given set of input tensors.

Inference evaluates the model using its current parameter values without performing optimizer-driven parameter updates.

The predict() method provides this interface.

```cpp
auto predicted_output = mymodel.predict({input});
```

Listing 9.4: Generating predictions using a trained model

The input tensors are evaluated through the model's computational graph, and the resulting output tensors are returned to the caller.

Unlike training, inference does not require target outputs or a loss calculation for the purpose of updating model parameters. Its primary responsibility is to produce predictions from the supplied inputs.

When predict() is called on a model that has already been trained, it uses the model's current weights and biases. Consequently, the predictions reflect the parameter updates performed during training.

Separating training from inference provides a clear distinction between learning the model parameters and using those parameters to evaluate new data.

9.2 Numerical Verification

Implementing the model API is only one part of developing a neural network framework. Its numerical behavior must also be verified against a trusted reference.

LearnDeep uses NumPy to generate reference results for selected operations and model configurations. These results can then be compared against the outputs produced by the C++ implementation.

For a forward-pass test, the model is configured with known weights and biases, and its predictions are compared with the corresponding NumPy results. This helps verify that the model correctly combines its underlying tensor operations.

Training requires additional verification. The parameter updates and loss behavior should be examined independently to establish that gradients and optimization are working as intended.

Finally, inference can be tested by checking whether predict() returns the expected output tensors for a known model state and a given set of inputs.

These tests address different aspects of correctness: forward-pass tests verify numerical computation, training tests examine gradient-based optimization, and inference tests verify the prediction interface.

Together, they provide a systematic approach to validating the integration of the components developed throughout this framework.

$$\nabla_\theta L$$